TDDE70 Lab 3 Prepatory exercises

Filip Ekström Kelvinius, Fredrik Lindsten

April 2026

1 Introduction

In this lab, you will practice using graph neural networks (GNNs) and implementing them with PyTorch. Specifically, the task you will try to solve with a GNN is predicting the energy of molecules, and more details of this is introduced in section 2. Then, in section 3, you will find the preparatory exercises. The aim is to start thinking about graphs and GNNs in general, but also that you start familiarizing yourself with implementations of these in PyTorch, and the type of data that you will work with in the lab.

NOTE: For implementing GNNs, we will use a package called `Pytorch Geometric`, and in particular its `MessagePassing` class. **For the lab, use the conda environment `lab3` (can be activated the same way as before, but using `conda activate lab3`). In case you are not using the lab computers, `lab3.yml` specifies the required packages.**

2 Machine learning for molecules

The data that you will work with is a molecular dataset. This section aims to give a brief introduction on this application domain and why it is important, how molecules can be represented as graphs so that we can apply GNNs to them, and a little background on the dataset you will use in the lab.

2.1 Why molecules?

One of the most prominent applications of machine learning for molecules is for drug discovery. Imagine that a team of researchers is looking for a novel drug to address a certain medical condition. Machine learning can help with this task in many different ways. One approach is to screen through a large pool of hypothetical drug candidates (for instance generated by a generative machine learning model) by letting a machine learning model predict various properties of interest for these drug candidates. The ones with desirable (predicted) properties can then be further examined and targets for being synthesized in the lab.

2.2 The task

In this lab we will train a GNN to predict the so-called atomization energy E of a molecule. Loosely speaking, in this context the atomization energy is the energy difference between the molecule and the individual atoms (i.e., if the molecule is broken up into its constituent atoms), and can be thought of as the energy that we would need to spend to break all the chemical bonds of the molecule and split it into individual atoms. Hence, the atomization energy is informative about the bonding strength of the molecule (how "strong" are the bonds holding the atoms together), and gives insights into, e.g., the overall stability of the molecule, which is a key aspect to consider if we actually want to synthesize the drug in a lab (if the molecule is unstable, it means it risks spontaneously "fall apart").

In essence, a molecule is a "cloud" of N atoms in space. This can mathematically be represented using N different 3-dimensional vectors with atom positions and a vector of N discrete atom types (atomic numbers). It is rather straightforward to construct a graph based on the coordinates: each atom corresponds to a node, and we can create edges between atoms that are within a certain distance from each other. An illustration in two dimensions of a toy molecule and how a graph could be constructed is shown in Figure 1.

In summary, the energy prediction problem can be described mathematically as

$$\hat{E} = \text{GNN}(\mathbf{X}, \mathbf{z}), \tag{1}$$

where $\mathbf{X} \in \mathbb{R}^{N \times 3}$ denotes the positions of each atom, and $\mathbf{z} \in \mathbb{Z}^N$ denoting the type (atomic number) of each atom. The graph $(\mathcal{V}, \mathcal{E})$ is constructed from the input $\mathbf{X}$ as explained above and in Figure 1. The initial node features $\{\mathbf{h}_i^0\}_{i=1}^N$ could then be (an embedding of) the type of atom, $\mathbf{h}_i^0 = \text{Embed}(\mathbf{z}_i)$, and edge features $\{\mathbf{e}_{(i,j)}\}_{(i,j)\in\mathcal{E}}$ could incorporate geometric information like the distance between the atoms.

2.3 The COLL dataset

In the lab we will work with a molecular dataset referred to as COLL [2]. This dataset has been constructed by running molecular dynamics simulations of molecular collisions. This means using computer simulations to compute forces acting on atoms, then move the atoms a little according to the forces, and then redo this many times. This creates a "trajectory" of how the molecule develops over time, and snapshots (the positions of the atoms, the forces, and the energy) from this trajectory constitute the dataset.[1]

[1] Although the trajectories have a temporal aspect, this is not something that is included in the dataset, and we will just view the different snapshots as independent data points.

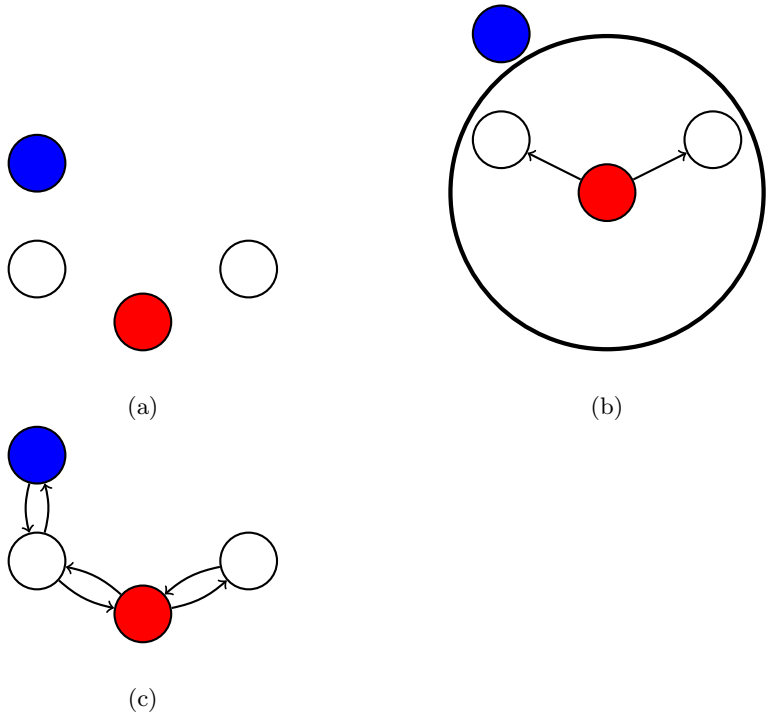

Figure 1: Illustration of how to construct a graph from a molecule. (a) A toy molecule in two dimensions. Different colors correspond to different types of atoms. (b) To construct edges, we can connect atoms within a certain distance from each other. In this example, we add edges from the red atom to atoms within a certain cutoff radius illustrated by the black circle. In three dimensions, the circle would correspond to a sphere. (c) By applying this process to the other atoms, we obtain our graph. In this case, the edges are symmetric in the sense that for each edge (i, j) there is always another edge (j, i). However, depending on how the molecule looks like and if we impose the additional constraint that an atom can only have a certain maximum number of neighbors, this might not be the case.

3 Preparation exercises

What is graph machine learning anyway?

In the lab, you will apply your GNN to the task of predicting properties of molecules, as these can nicely be modeled as graphs. However, this is far from the only type of data that can be described as graphs. In general, a graph G is defined by a set of nodes, or vertices, $\mathcal{V}$, and a set of edges $\mathcal{E}$.

Question 1 *Can you give some other examples of data which are suitable to represent as graphs?*

Question 2 *Imagine you are working with the type of data from the previous question, what tasks could you try to solve with a graph neural network?*

Question 3 *Are those tasks* node *or* graph *level tasks, or maybe some other level?*

Question 4 *Is the atomization energy prediction task that we will address in the lab a* node *or* graph *level task?*

Deriving MPNN equations

`Pytorch Geometric` provides support for the message passing neural network (MPNN) framework that you saw in the lectures. Therefore, it is rather straightforward to implement a GNN if you have derived the functions M_l and U_l from the MPNN framework.

In this lab, you will work with a GNN called CGCNN [1] and which has been developed for chemistry applications. To implement this model in `Pytorch Geometric`, we will start by deriving the MPNN equations for CGCNN. In the original paper, they use the equation (compare eq. 5 in the paper)

$$\mathbf{h}_i^{(l+1)} = \mathbf{h}_i^{(l)} + \sum_{j \in \mathcal{N}(i)} \sigma(W_f^{(l)} \mathbf{z}_{i,j}^{(l)} + b_f^{(l)}) \odot g(W_s^{(l)} \mathbf{z}_{i,j}^{(l)} + b_s^{(l)}) \tag{2}$$

to describe their updates to the node features $\mathbf{h}_i$. Here, W and b denote weight matrices and vectors, respectively, $\sigma()$ and $g()$ are functions operating element-wise,[2] $\odot$ means element-wise multiplication, and $\mathbf{z}_{i,j} = \mathbf{h}_i \oplus \mathbf{h}_j \oplus \mathbf{e}_{i,j}$ is the concatenation of node and edge feature vectors.

Question 5 *Given the equation above, re-write this according to the MPNN-equations from the lectures, i.e., write down the functions* $M_l(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}, \mathbf{e}_{i,j})$ *and* $U_l(\mathbf{h}_i^{(l)}, \mathbf{m}_i^{(l)})$.

Question 6 *To what "flavor" of GNNs (see slides, or the GDL [3] book chapter 5.3) does CGCNN belong to? Include a short motivation*

[2] In the paper, it is mentioned that $\sigma(\cdot) = \text{sigmoid}(\cdot)$ implements a *gating* mechanism: as the output of sigmoid is limited to be between 0 and 1, it essentially determines how much of each channel of the output of $g(\cdot)$ that is kept or discarded.

```python
import torch
import torch.nn as nn
import torch_geometric as pyg

class GNNLayer(pyg.nn.MessagePassing):
    def __init__(self, h_dim, edge_dim):
        super().__init__(aggr="sum")
        self.layer_i = nn.Linear(h_dim, h_dim)
        self.layer_j = nn.Linear(h_dim, h_dim)
        self.layer_edge = nn.Linear(edge_dim, h_dim)

        self.layer_u = nn.Linear(2*h_dim, h_dim)

        self.sigma = nn.Softplus()

    def forward(self, h, edge_attr, edge_index):
        m = self.propagate(edge_index, h=h, e=edge_attr)
        hm_cat = torch.cat([h, m], dim=-1)
        out = h + self.sigma(self.layer_u(hm_cat))
        return out

    def message(self, h_i, h_j, e):
        out_i = self.sigma(self.layer_i(h_i))
        out_j = self.sigma(self.layer_j(h_j))
        out_e = self.sigma(self.layer_edge(e))
        return out_i * out_j * out_e
```

Listing 1: Example of a GNN-layer implemented in `Pytorch Geometric`

Implementation of MPNN equations

Now, let's take a look at how a set of MPNN equations can be implemented in `Pytorch Geometric`. First, have a look at the `Pytorch Geometric` intro tutorial,[3] in particular the introduction, the part about the `MessagePassing` base class, and about implementing GCN. **Note:** the `update` function has according to the developers been phased out, and U_l can instead be implemented inside `forward`, where $\mathbf{m}_i^{(l)}$ is the output of `propagate`.

Question 7 *Look at the code in Listing 1 above, and write down the corresponding MPNN equations for this layer.*

Something about invariance

The task for the GNN in the lab is to predict the energy of molecules. This property is *invariant* to any rigid transformations (i.e., rotations and/or translations) of the molecules; it does not matter if we move the molecule to a different position, and/or if it is rotated, it is still the same molecule with the same energy, see Figure 2.

[3] `https://pytorch-geometric.readthedocs.io/en/latest/notes/create_gnn.html`

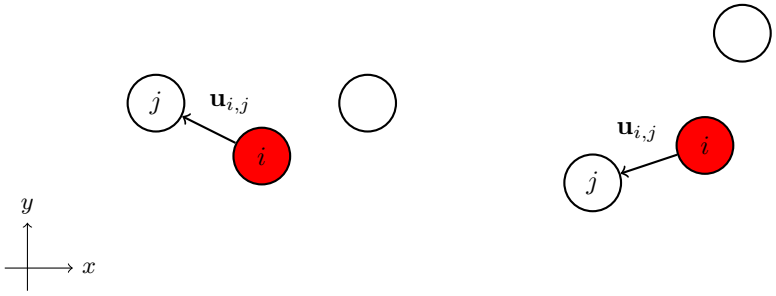

Figure 2: A water molecule in 2D. The right molecule is the same water molecule, but translated and rotated, and hence has the same energy. The vector $\mathbf{u}_{i,j}$ is the vector between the two atoms i and j.

We can make our network invariant by construction to such transformations by using *only invariant features*. The input to CGCNN consists of the atom types $\mathbf{z}$ (node features) and the atom positions $\mathbf{X}$. The atom types are of course invariant to translations and rotations, so it is the geometric information in $\mathbf{X}$ that needs consideration. CGCNN incorporates this geometric information in the edge features.

Question 8 *If denoting $\mathbf{u}_{i,j}$ the vector between atom i and j (see Figure 2), consider using as edge feature $\mathbf{e}_{i,j}$ either 1) the distance between the two atoms ($\mathbf{e}_{i,j} = ||\mathbf{u}_{i,j}||$) or 2) the vector between the atoms ($\mathbf{e}_{i,j} = \mathbf{u}_{i,j}$). To what transformations (rotations, translations) are these respective features invariant?*

A final remark

In Appendix A you can find a block diagram that illustrates the CGCNN model. This can serve as some help when you implement this model in the lab.

References

[1] Tian Xie and Jeffrey C. Grossman. "Crystal Graph Convolutional Neural Networks for an Accurate and Interpretable Prediction of Material Properties". In: *Phys. Rev. Lett.* 120 (14 Apr. 2018), p. 145301. DOI: 10.1103/PhysRevLett.120.145301. URL: https://link.aps.org/doi/10.1103/PhysRevLett.120.145301.

[2] Johannes Gasteiger et al. "Fast and Uncertainty-Aware Directional Message Passing for Non-Equilibrium Molecules". In: *Machine Learning for Molecules Workshop, NeurIPS*. 2020.

[3] Michael M. Bronstein et al. *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges*. 2021. arXiv: 2104.13478 [cs.LG]. URL: https://arxiv.org/abs/2104.13478.

A CGCNN Block Diagram

Figure 3 shows a block diagram for the CGCNN model that you can use as help when implementing your CGCNN model.

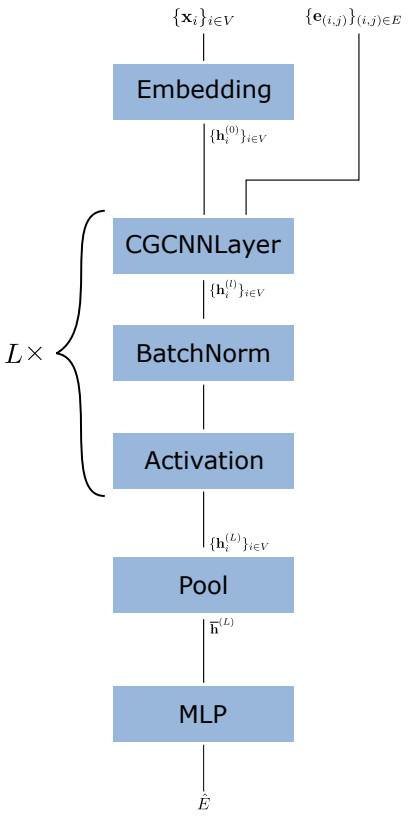

Figure 3: Block diagram for the CGCNN model that you will implement in the lab.